

Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. INTRODUCTION

STM32 is a family of microcontrollers from STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

Sensor applications perform recurring tasks, such as reading temperature, measuring distance, capturing images, and acquiring audio. These tasks may differ in their periods and execution times. A long-running task can delay other tasks, even when their individual workloads are small. Scheduling must therefore account for when jobs become ready and when they must complete.

SuperLoop repeatedly calls software components within a main loop [2]. Our periodic implementation polls tasks in a fixed order and executes each selected job to completion. Earliest Deadline First (EDF) instead selects the ready job with the earliest absolute deadline [3]. We implement chunked EDF, which repeats job selection at chunk boundaries. The chunk duration determines how long an earlier-deadline job may wait and how often the scheduler makes a selection.

A. Contribution

We compare periodic SuperLoop and chunked EDF on an STM32F767 using synthetic workloads. We evaluate the effects of workload utilization and chunk duration on response time, deadline misses, skipped releases, and measured selection overhead. Our implementations for both schedulers are publicly available.

II. BACKGROUND ON STM32

We illustrate the hardware organization for our system in Fig. 1. The central processing unit is an STM32F767 uniprocessor, which in turn is connected to four sensors:

- DHT11: to measure temperature and humidity [4];
- HC-SR04: to measure distance [5];
- OV2640: to capture images [6];
- SPW2430: to acquire acoustic signal [7].

III. SYSTEM MODEL

We rely on a classical real-time periodic system model by Liu and Layland [3]. We adopt the task and release notation of Burmyakov and Bini [8].

A real-time system $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$ consists of n periodic tasks executed on a uniprocessor. Each task $\tau_i = (C_i, D_i, P_i)$ has a worst-case execution time C_i , a relative deadline D_i , and a period P_i . We consider implicit deadlines, $D_i = P_i$. Task

utilization is $U_i = C_i/P_i$, and task-set utilization is $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$.

Let $J_{i,j}$ denote the j -th job of τ_i , released at $r_{i,j}$. Unlike the sporadic model in the referenced paper, consecutive releases are separated by exactly P_i :

$$r_{i,j+1} = r_{i,j} + P_i. \quad (1)$$

Job $J_{i,j}$ requires at most C_i units of CPU time and has an absolute deadline $r_{i,j} + D_i$. Its completion time, when executed to completion, is denoted by $f_{i,j}$. Release indices include jobs skipped by the implementation, these have no completion time.

TABLE I
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j -th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Completion time of an executed job
$r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
$U_i = C_i/P_i$	Task utilization
$U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ has missed its deadline if $f_{i,j} > r_{i,j} + D_i$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$, the time from its release to completion.

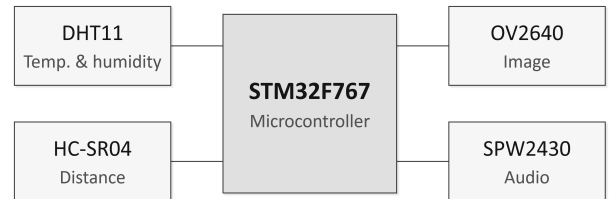


Fig. 1. Our experimental hardware platform.

Table I summarizes the notation introduced above. The evaluation plots use U as shorthand for $U_{\mathcal{T}}$.

IV. SCHEDULERS

We describe the periodic SuperLoop and chunked EDF schedulers used in our evaluation.

A. SuperLoop

Algorithm 1 SuperLoop Scheduler

```

1: function SUPERLOOP( $\mathcal{T}$ )
2:    $t_i^{\text{next}} \leftarrow 0, \quad i = 1, \dots, n$   $\triangleright$  Initial releases
3:   while true do
4:     for  $i \leftarrow 1$  to  $n$  do  $\triangleright$  Fixed task order
5:        $t^{\text{cur}} \leftarrow \text{NOW}$ 
6:       if  $t_i^{\text{next}} \leq t^{\text{cur}}$  then
7:          $J_{i,j} \leftarrow \text{RELEASEJOB}(\tau_i, t_i^{\text{next}})$ 
8:         EXECUTE( $J_{i,j}$ )  $\triangleright$  Non-preemptive
9:          $f_{i,j} \leftarrow \text{NOW}$   $\triangleright$  Completion time
10:         $t_i^{\text{next}} \leftarrow t_i^{\text{next}} + P_i$ 
11:        while  $t_i^{\text{next}} < f_{i,j}$  do  $\triangleright$  Equality retained
12:           $t_i^{\text{next}} \leftarrow t_i^{\text{next}} + P_i$ 
13:        end while
14:      end if
15:    end for
16:  end while
17: end function

```

The considered SuperLoop implementation polls tasks in a fixed order and executes each ready job to completion. Algorithm 1 outlines its key steps. To illustrate its behavior, we consider the two periodic tasks in Table II. Both tasks first release at $t = 0$.

TABLE II
TASK SET USED IN THE SUPERLOOP EXAMPLE.

Task	C_i	P_i	D_i
τ_1	20	50	50
τ_2	10	20	20

SuperLoop

$$\tau_i = (C_i, D_i, P_i), D_i = P_i$$

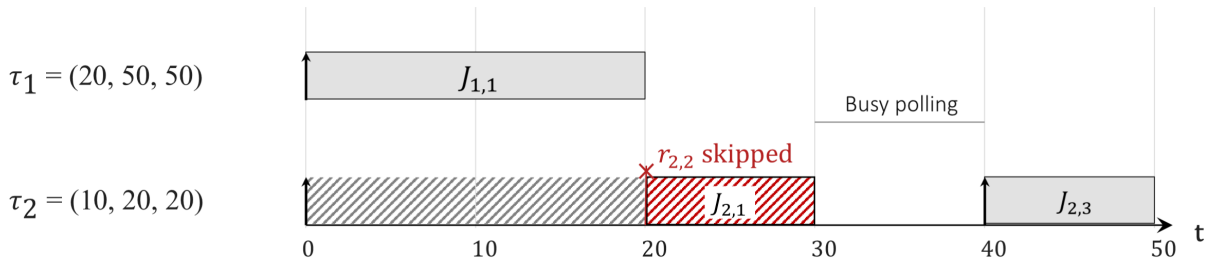


Fig. 2. SuperLoop schedule for Table II. Arrows mark releases and the cross marks a skipped release. Gray hatching indicates waiting and red hatching indicates execution after the deadline.

The resulting SuperLoop schedule is shown in Fig. 2.

Let t^{cur} denote the current system time and t_i^{next} the next release time tracked for task τ_i . During execution of $J_{i,j}$, t_i^{next} retains $r_{i,j}$. At completion, the scheduler advances t_i^{next} by P_i . While $t_i^{\text{next}} < f_{i,j}$, the corresponding release is skipped and t_i^{next} is advanced by P_i again. A release at $t_i^{\text{next}} = f_{i,j}$ is retained.

In Algorithm 1, RELEASEJOB creates $J_{i,j}$ with $r_{i,j} = t_i^{\text{next}}$ and $j = 1 + t_i^{\text{next}}/P_i$, including skipped releases. The clock used by NOW starts at zero. Metric collection is omitted from both algorithms.

The corresponding scheduling state is shown in Table III.

TABLE III
SUPERLOOP SCHEDULING STATE FOR FIG. 2.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	—	—
20	50	0	$20 < r_{1,1} + 50$	$20 < r_{1,2}$
30	50	40	$30 > r_{2,1} + 20$	$30 > r_{2,2}$
40	50	40	—	—
50	50	60	$50 < r_{2,3} + 20$	$50 < r_{2,4}$

In Fig. 2, $J_{1,1}$ executes from $t = 0$ to $t = 20$. Then, $J_{2,1}$ executes until $t = 30$, missing its deadline at $t = 20$. At completion, the job released at $r_{2,2} = 20$ is counted as skipped and t_2^{next} advances to $r_{2,3} = 40$. Thus, no $J_{2,2}$ is executed.

B. Earliest Deadline First

Unlike SuperLoop, Earliest Deadline First (EDF) selects jobs according to their absolute deadlines.

We first consider preemptive EDF with immediate preemption. At each scheduling decision, the ready job with the earliest absolute deadline $r_{i,j} + D_i$ is selected (Section III). If a newly released job has an earlier absolute deadline than the running job, it preempts the running job.

Fig. 3 illustrates EDF scheduling for the task set in Table II.

At $t = 20$, $J_{2,2}$ is released with $r_{2,2} + D_2 = 40 < r_{1,1} + D_1 = 50$ and preempts $J_{1,1}$. After $J_{2,2}$ completes at $t = 30$, $J_{1,1}$ resumes and completes at $t = 40$.

EDF

$$\tau_i = (C_i, D_i, P_i), D_i = P_i$$

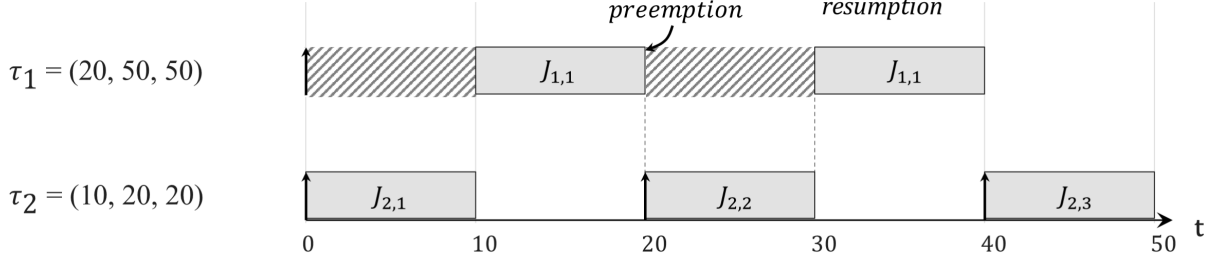


Fig. 3. EDF schedule for Table II. Arrows mark releases and hatching indicates waiting.

Chunked EDF is used as a simplified prototype implementation, avoiding the additional complexity of full task-context switching during initial evaluation. Synthetic jobs execute in chunks of maximum duration Δ , with job selection repeated after each chunk. An earlier-deadline job released during a chunk may therefore wait until the chunk completes. Sensor jobs use separate staged handlers.

For illustration, Fig. 4 uses $\Delta = 4$ ms. Job $J_{2,2}$ is released at $t = 20$ ms and preempts $J_{1,1}$ at the next chunk boundary $t = 22$ ms. Thus, the preemption delay is $\delta = 2$ ms. The final chunk of $J_{2,2}$ lasts 2 ms, since the preceding two chunks account for 8 ms of its execution time $C_2 = 10$ ms.

Ignoring scheduling overhead, the delay until the next chunk boundary satisfies $0 \leq \delta \leq \Delta$.

Algorithm 2 summarizes chunked EDF for synthetic jobs, assuming first releases at $t = 0$. The set $\mathcal{R}$ contains active jobs, meaning jobs admitted for execution but not yet completed, including preempted jobs. Initially, $\mathcal{R} = \emptyset$, so no job is active. Equal deadlines are resolved by a fixed task order, the three-task experiments use τ_2, τ_1, τ_3 . EXECUTECHUNK executes at most Δ of the selected job's remaining execution and preserves its progress.

The auxiliary operations are:

- RELEASEJOB: creates and returns the job as in Algorithm 1, sets its absolute deadline to $r_{i,j} + D_i$, and additionally adds it to $\mathcal{R}$.
- COMPLETEJOB: advances t_i^{next} using the skip rule in Algorithm 1 and removes the completed job from $\mathcal{R}$.

Algorithm 2 Chunked EDF Scheduler

```

1: function CHUNKEDEDF( $\mathcal{T}, \Delta$ )
2:    $t_i^{\text{next}} \leftarrow 0, \quad i = 1, \dots, n$   $\triangleright$  Initial releases
3:   while true do
4:      $t^{\text{cur}} \leftarrow \text{NOW}$ 
5:     for all  $\tau_i \in \mathcal{T}$  do  $\triangleright$  Check releases
6:       if  $t_i^{\text{next}} \leq t^{\text{cur}}$  and  $\tau_i$  has no active job then
7:         RELEASEJOB( $\tau_i, t_i^{\text{next}}$ )  $\triangleright$  Add to  $\mathcal{R}$ 
8:       end if
9:     end for
10:    if  $\mathcal{R} \neq \emptyset$  then  $\triangleright$  Pending work
11:       $J_{i,j} \leftarrow \text{SELECTEDF}(\mathcal{R})$   $\triangleright$  Earliest deadline
12:      EXECUTECHUNK( $J_{i,j}, \Delta$ )  $\triangleright$  At most  $\Delta$ 
13:      if COMPLETED( $J_{i,j}$ ) then
14:         $f_{i,j} \leftarrow \text{NOW}$ 
15:        COMPLETEJOB( $J_{i,j}, f_{i,j}$ )  $\triangleright$  Skip rule
16:      end if
17:    end if
18:  end while
19: end function

```

V. PERFORMANCE EVALUATION

A. Experimental setup

Experiments run on the NUCLEO-F767ZI at 96 MHz. Physical sensors are disabled in these experiments. Each synthetic job executes a timed busy-wait workload, the scheduler also

Chunked EDF

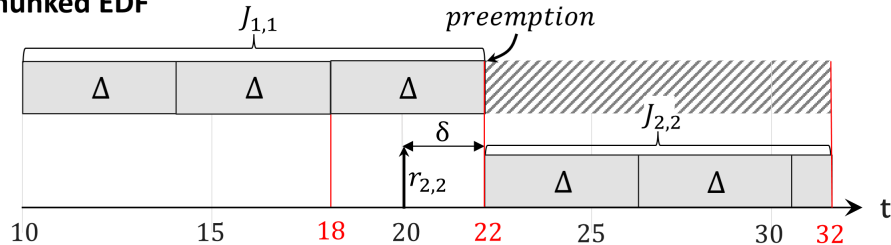


Fig. 4. Chunk-bound preemption in the EDF implementation.

uses busy polling. Timing is obtained from the DWT cycle counter, and UART reports are emitted after measurement stops.

The results below use three tasks with $(P_1, P_2, P_3) = (10, 50, 200)$ ms and $D_i = P_i$. Table IV lists the configured synthetic workload durations, which are nominal budgets rather than measured worst-case bounds. Thus, $U_{\mathcal{T}}$ denotes nominal workload utilization and excludes instrumentation and scheduling overhead. Both schedulers use the same task set, SuperLoop polls tasks in index order. For clarity, the illustrative schedules assume synchronous first releases at $t = 0$, whereas the implementation releases each task for the first time one period after initialization, $r_{i,1} = t_0 + P_i$.

TABLE IV

THREE-TASK SYNTHETIC WORKLOADS. EXECUTION BUDGETS ARE IN μs .

Nominal $U_{\mathcal{T}}$ (%)	C_1	C_2	C_3
50	1667	11111	22222
65	2167	14444	28889
75	2500	16667	33332
80	2667	17778	35556
90	3000	20000	40000
95	3167	21111	42222
100	3333	22222	44444

The main comparison uses 60 s windows and $\Delta = 1$ ms for EDF. The chunk-size study uses $\Delta \in \{1, 2, 4\}$ ms at $U_{\mathcal{T}} \in \{50, 90, 100\}\%$, the figure below shows $U_{\mathcal{T}} = 90\%$. Each configuration is run three times from reset. Additional campaigns use two tasks and 100 s windows to examine task-set and window-length effects. Extended diagnostic statistics are disabled in all these campaigns.

B. Evaluation metrics

Let N_i , M_i , and S_i be the numbers of completed jobs, completed jobs that missed their deadlines, and skipped releases of τ_i in a run. Let W be the requested measurement-window duration. We report:

- Deadline-miss ratio: $100M_i/N_i$ percent, defined for $N_i > 0$.
- Skipped-release ratio: $100S_i/(W/P_i)$ percent, normalized by the nominal number of releases in the requested window.
- Average response time: the mean of $f_{i,j} - r_{i,j}$ over completed jobs. The maximum observed response time is also recorded.
- Selection overhead: measured time spent selecting jobs in iterations that execute work, divided by the actual measurement duration, in percent. This excludes uninstrumented scheduler work and is not total processor utilization.

Measurement stops at the first loop boundary after W , its actual duration may therefore be slightly longer. Released but unfinished tracked jobs and their overdue status are recorded separately and excluded from completion-based metrics. Subsequent releases awaiting skip accounting are not included in

this pending-job count. Plot points are arithmetic means of the corresponding per-run metrics over three runs, not pooled averages over all jobs.

C. SuperLoop and EDF comparison

Fig. 5 compares the schedulers for τ_1 . Under SuperLoop, its deadline-miss and skipped-release ratios generally increase with utilization, as does its average response time. With chunked EDF ($\Delta = 1$ ms), no deadline misses or skipped releases are recorded for τ_1 in the evaluated three-task configurations. Selecting jobs by absolute deadline reduces the blocking caused by the fixed SuperLoop order.

Fig. 6 shows the corresponding results for τ_3 . Its average response time under chunked EDF increases with utilization as jobs with earlier absolute deadlines execute first. At nominal $U_{\mathcal{T}} = 100\%$, no capacity is reserved for scheduling and execution overhead, deadline misses and skipped releases are observed for τ_3 . The lower response time of τ_3 under SuperLoop occurs alongside skipped releases of τ_1 , so it does not imply better service for the entire task set.

D. Effect of chunk duration

Fig. 7 examines Δ at $U_{\mathcal{T}} = 90\%$. Larger chunks reduce the frequency of scheduling decisions. In these measurements, the reported selection overhead decreases, while the average response time of τ_1 increases and that of τ_3 decreases. Longer non-preemptive intervals delay τ_1 , the result for τ_3 also reflects the changed execution order and fewer scheduling interruptions. The overhead measurements remain preliminary pending validation of the instrumentation.

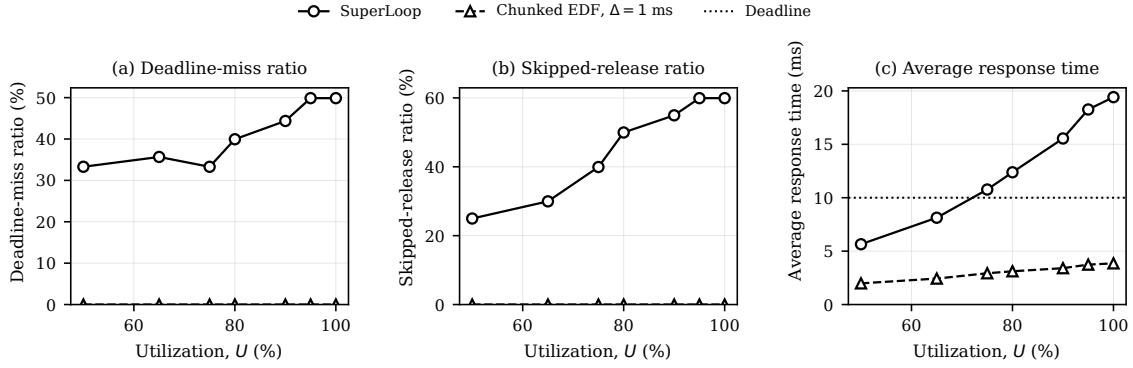


Fig. 5. Performance for τ_1 : three synthetic tasks, $\Delta = 1$ ms for EDF, 60 s windows.

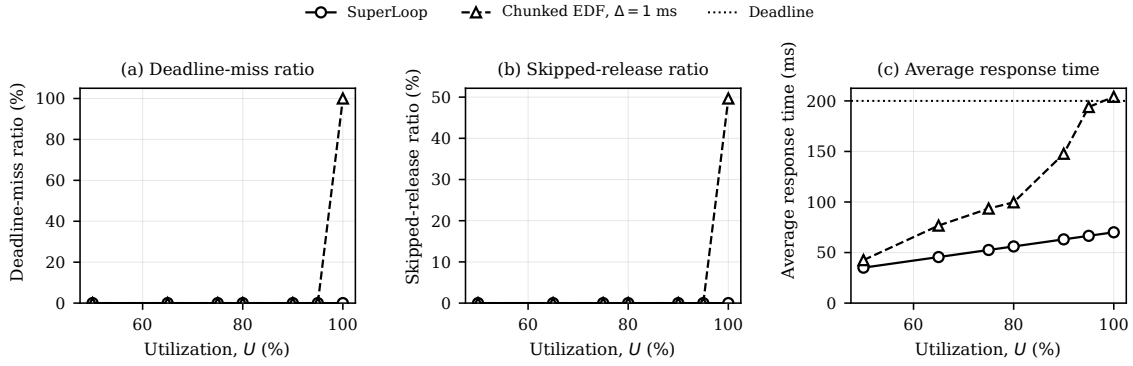


Fig. 6. Performance for τ_3 under the same conditions as Fig. 5.

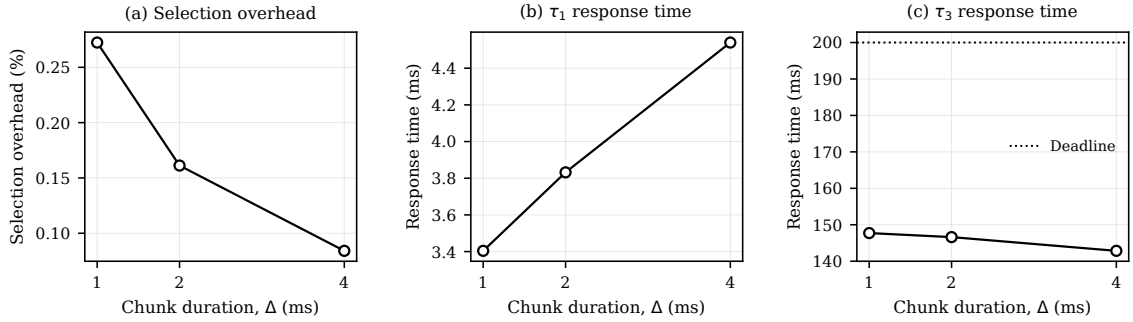


Fig. 7. Impact of chunk duration at $U_T = 90\%$ with three synthetic tasks and 60 s windows.

REFERENCES

- [1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f767zi.pdf>
- [2] Arm Keil, “Execution models: Single task system (Superloop),” accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html
- [3] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [4] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>
- [5] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [6] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf
- [7] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: <https://www.ibttkar.io/Robotics/SPW2430HR5H-B.pdf>
- [8] A. Burmyakov and E. Bini, “Improving the efficiency of global EDF exact schedulability test,” *IEEE Transactions on Computers*, pp. 1–14, 2026.